

Spécifications du formateur AutoLISP

OpenAI

April 27, 2026

Contents

1	Objet du projet	4
2	Périmètre fonctionnel	4
2.1	Fonction principale	4
2.2	Hors périmètre initial	5
3	Contraintes majeures	5
3.1	Langage d'implémentation	5
3.2	Fidélité source	5
3.3	Déterminisme	5
3.4	Idempotence	5
3.5	Tolérance au code existant	6
4	Architecture générale	6
4.1	Passe 1 : lecture brute du fichier	6
4.2	Passe 2 : scanner / tokenizer	6
4.3	Passe 3 : parseur syntaxique léger	7
4.4	Passe 4 : normalisations optionnelles	7
4.5	Passe 5 : pretty-printer	7
5	Modèle syntaxique minimal à supporter	7
5.1	Éléments lexicaux	7
5.1.1	Paranthèses	7
5.1.2	Quote	7
5.1.3	Chaînes	7
5.1.4	Commentaires de ligne	8
5.1.5	Commentaires de bloc	8
5.1.6	Atomes	8

5.1.7	Dotted pairs	9
5.2	S-expressions à supporter	9
5.3	Formes connues à mieux indenter	9
6	Casse des symboles	10
6.1	Principe	10
6.2	Options requises	10
6.3	Règles de transformation	10
6.4	Remarque sur :foo	10
7	Styles de parenthèses	10
7.1	Style <code>cl</code>	11
7.2	Style <code>nail</code>	11
7.3	Contraintes	11
8	Commentaires	11
8.1	Modes de gestion	11
8.1.1	Mode <code>preserve</code>	12
8.1.2	Mode <code>cl</code>	12
8.2	Conversion des commentaires bloc	12
8.3	Conversion contextuelle	12
8.4	Alignement des commentaires de fin de ligne	13
9	Préservation et transformations	13
9.1	À préserver impérativement	13
9.2	Transformations autorisées	13
10	Options de ligne de commande / d'appel	14
10.1	Options fonctionnelles minimales	14
10.2	Sémantique attendue	14
11	Stratégie de représentation interne	14
12	Règles de qualité	15
12.1	Déterminisme	15
12.2	Idempotence	15
12.3	Non-régression syntaxique	15
12.4	Lisibilité	15

13 Gestion des erreurs	16
13.1 Erreurs détectables	16
13.2 Comportement attendu	16
14 Jeux de tests requis	16
14.1 Tests unitaires scanner	16
14.2 Tests unitaires parseur	17
14.3 Golden tests pretty-printer	17
14.4 Tests d'idempotence	17
14.5 Tests de non-régression sur corpus réel	17
15 Plan de livraison recommandé	17
15.1 Version 1	17
15.2 Version 2	18
16 Critères d'acceptation	18
17 Style supplémentaire de parenthèses : nail-clipping à fermetures empilées	19
17.1 Parenthesis-style enumeration update	20
17.2 Validation rule	20
17.3 Naming rationale	20
18 Exceptions de casse par fichier de symboles	21
18.1 Format du fichier d'exceptions	21
18.2 Sémantique	21
18.3 Priorité des règles de casse	22
18.4 Options associées	22
19 Fichier de configuration du formateur	23
19.1 Objectif	23
19.2 Formes acceptées	23
19.3 Correspondance programme / API	24
19.4 Résolution des conflits	24
19.5 Options minimales à prévoir	24
19.6 Recommandation de mise en oeuvre	25

1 Objet du projet

Ce projet vise à réaliser un **formateur** / **pretty-printer** **AutoLISP** écrit en **AutoLISP**, destiné à un usage d'équipe sur des bases de code existantes et nouvelles.

Objectifs principaux :

- reformater le code AutoLISP de manière **déterministe** ;
- corriger automatiquement l'indentation et le placement des parenthèses ;
- préserver ou normaliser les commentaires selon la politique choisie ;
- proposer plusieurs styles configurables pour faciliter l'adoption progressive ;
- fonctionner sans dépendance externe autre que l'environnement AutoLISP / VisualLISP disponible côté équipe.

Le projet ne vise pas à implémenter un compilateur, un évaluateur, ni un lecteur sémantique complet d'AutoLISP. Il vise un **lecteur syntaxique source-à-source** capable de lire, restructurer et réémettre du code sans le dénaturer inutilement.

2 Périmètre fonctionnel

2.1 Fonction principale

Le programme prend en entrée un ou plusieurs fichiers source AutoLISP et produit une version reformattée selon un jeu d'options.

Modes d'utilisation minimaux attendus :

- formater un fichier vers la sortie standard ou un fichier de sortie ;
- formater un fichier **in place** ;
- vérifier si un fichier est déjà au bon format sans le modifier ;
- choisir une politique de style par options explicites.

2.2 Hors périmètre initial

Les fonctions suivantes ne sont pas requises en version 1 :

- reflow automatique du texte en prose dans les commentaires ;
- compréhension sémantique profonde des macros utilisateur ;
- refactoring de noms ;
- analyse de portée avancée ;
- intégration IDE sophistiquée ;
- conversion complète de styles de commentaires avec compréhension éditoriale parfaite ;
- conservation pixel-perfect de toute la mise en page d'origine.

3 Contraintes majeures

3.1 Langage d'implémentation

Le formateur doit être écrit en **AutoLISP**.

3.2 Fidélité source

Le système doit préserver la structure sémantique du source. Les chaînes de caractères et le texte interne des commentaires ne doivent pas être modifiés sauf option explicite de conversion de commentaires.

3.3 Déterminisme

À options égales, le même fichier d'entrée doit toujours produire exactement la même sortie.

3.4 Idempotence

Le programme doit être idempotent : reformater un fichier déjà formaté ne doit pas produire une nouvelle différence.

3.5 Tolérance au code existant

Le programme doit accepter du code historiquement mal indenté, notamment le style dit *nail-clipping* où les parenthèses fermantes sont placées sur des lignes séparées.

4 Architecture générale

Le système doit être structuré en passes distinctes.

4.1 Passe 1 : lecture brute du fichier

- lecture du fichier en texte brut ;
- conservation de l'ordre exact des caractères.

4.2 Passe 2 : scanner / tokenizer

Le scanner transforme le texte en une séquence de tokens, en distinguant au minimum :

- parenthèse ouvrante ;
- parenthèse fermante ;
- quote ;
- chaîne ;
- atome / symbole / nombre ;
- commentaire de ligne ;
- commentaire bloc ;| ... | ; ;
- espaces / tabulations ;
- fin de ligne.

Le scanner doit reconnaître les commentaires bloc AutoLISP de forme ;| ... | ;. Ceci diffère du Common Lisp qui utilise #| ... |#.

4.3 Passe 3 : parseur syntaxique léger

Le parseur reconstruit une représentation structurée du programme permettant le pretty-printing. Il n'a pas besoin d'évaluer les formes.

Cette représentation sera de préférence un **CST** (Concrete Syntax Tree) ou une structure équivalente préservant mieux la surface syntaxique qu'un AST purement sémantique.

4.4 Passe 4 : normalisations optionnelles

Passes configurables pour :

- changement de style de parenthèses ;
- normalisation de casse des symboles ;
- normalisation ou conversion de commentaires.

4.5 Passe 5 : pretty-printer

Réémission textuelle déterministe du programme selon les options choisies.

5 Modèle syntaxique minimal à supporter

5.1 Éléments lexicaux

Le scanner doit supporter correctement les éléments suivants.

5.1.1 Parenthèses

- (
-)

5.1.2 Quote

- ' ,

5.1.3 Chaînes

Chaînes délimitées par doubles guillemets, avec leur contenu préservé. Les parenthèses apparaissant dans les chaînes ne doivent pas affecter le comptage des parenthèses.

5.1.4 Commentaires de ligne

Tout ce qui suit ; jusqu'à la fin de ligne est un commentaire de ligne, sauf dans le cas particulier de l'ouverture d'un commentaire bloc ;|.

5.1.5 Commentaires de bloc

- ouverture : ;|
- fermeture : |;

Les parenthèses contenues dans un commentaire bloc ne doivent pas affecter l'analyse structurale.

5.1.6 Atomes

Le formateur doit accepter des symboles AutoLISP comportant notamment :

- lettres ;
- chiffres ;
- tirets ;
- astérisques ;
- points d'exclamation ;
- slash ;
- deux-points ;
- préfixes vl-, vlax-, vla-, etc.

Exemples légitimes à supporter :

- pk!
- vl-catch-all-apply
- *debug*
- :foo
- /

5.1.7 Dotted pairs

Le parseur doit au minimum ne pas casser les formes contenant un point de paire pointée, par exemple (a . b).

5.2 S-expressions à supporter

Le programme doit savoir structurer correctement au minimum :

- listes ordinaires ;
- atomes ;
- quote simple ;
- listes imbriquées ;
- formes top-level successives ;
- commentaires intercalés.

5.3 Formes connues à mieux indenter

La vl doit prévoir des règles de style dédiées au moins pour :

- defun
- lambda
- if
- cond
- setq
- progn
- while
- repeat
- foreach
- quote

Des règles spécifiques additionnelles pourront être ajoutées ensuite.

6 Casse des symboles

6.1 Principe

AutoLISP est traité ici comme **case-insensitive** pour les symboles dans l'usage courant. Le formateur doit proposer une politique de casse configurable.

6.2 Options requises

`:preserve` préserver la casse d'origine ;
`:downcase` convertir tous les symboles en minuscules ;
`:upcase` convertir tous les symboles en majuscules.

6.3 Règles de transformation

La transformation de casse ne s'applique qu'aux **tokens symbole**.

Elle ne doit jamais modifier :

- les chaînes ;
- le texte des commentaires ;
- les nombres ;
- les espaces ;
- la ponctuation structurelle.

6.4 Remarque sur `:foo`

Les formes `:foo`, `:bar`, etc. ne sont pas des *keywords* au sens Common Lisp ; ce sont des symboles ordinaires dont le nom commence par `:`. Elles doivent donc être traitées comme des symboles normaux.

7 Styles de parenthèses

Le formateur doit proposer au moins deux styles.

7.1 Style c1

Toutes les parenthèses fermantes sont ramenées sur la dernière ligne possible, selon l’usage Lisp classique.

Exemple :

```
(defun foo (x)
  (if (> x 0)
      (progn
        (bar x)
        (baz x)))))
```

7.2 Style nail

Le style *nail-clipping* maintient des parenthèses fermantes sur des lignes séparées, selon une convention compatible avec les habitudes d’équipes AutoLISP historiques.

Exemple :

```
(defun foo (x)
  (if (> x 0)
      (progn
        (bar x)
        (baz x)
      )
    )
  )
)
```

7.3 Contraintes

- les deux styles doivent être déterministes ;
- le style choisi doit être appliqué de manière cohérente à tout le fichier ;
- le formateur doit pouvoir reformater d’un style vers l’autre.

8 Commentaires

8.1 Modes de gestion

Le formateur doit proposer au moins deux politiques.

8.1.1 Mode `preserve`

Les commentaires sont préservés autant que possible tels quels.

Objectif : minimiser les surprises et faciliter l’adoption initiale.

8.1.2 Mode `c1`

Normalisation vers la convention inspirée de Common Lisp :

- `;;;;` pour commentaires de fichier / bannière ;
- `;;;` pour commentaires de section ;
- `;;` pour commentaires de code sur ligne dédiée ;
- `;` pour commentaire de fin de ligne.

8.2 Conversion des commentaires bloc

Le formateur doit proposer une option spéciale pour gérer les commentaires
`;| ... |;`.

Modes minimaux :

`preserve` conserver les commentaires bloc tels quels ;

`to-semicolons` convertir un commentaire bloc en série de commentaires préfixés par des points-virgules.

8.3 Conversion contextuelle

Une option contextuelle doit permettre de choisir le préfixe cible selon l’emplacement du commentaire :

- top-level isolé : `;;;;` ou `;;;` ;
- commentaire de bloc local : `;;;` ;
- commentaire en fin de ligne : `;`.

Il s’agit d’une heuristique. Une perfection éditoriale n’est pas exigée en v1.

8.4 Alignement des commentaires de fin de ligne

Une option doit permettre d'aligner les commentaires de fin de ligne sur une colonne donnée.

Exemples d'options :

- `nil` : ne pas forcer l'alignement ;
- `40` ;
- `48`.

Règle :

- si le code avant commentaire dépasse déjà la colonne cible, insérer au moins un espace avant le ; ;
- sinon, ajouter le nombre d'espaces nécessaire pour atteindre la colonne.

9 Préservation et transformations

9.1 À préserver impérativement

Sauf option contraire explicite, le formateur doit préserver :

- le texte des chaînes ;
- le texte interne des commentaires ;
- l'ordre sémantique des formes ;
- la structure logique du programme.

9.2 Transformations autorisées

Selon les options, il peut :

- modifier l'indentation ;
- déplacer les parenthèses fermantes ;
- normaliser la casse des symboles ;
- convertir les commentaires bloc ;
- aligner les commentaires de fin de ligne ;
- compacter ou stabiliser certaines lignes blanches selon la politique retenue.

10 Options de ligne de commande / d'appel

La forme exacte dépendra du lanceur retenu, mais l'outil doit exposer les concepts suivants.

10.1 Options fonctionnelles minimales

- `--style cl|nail`
- `--symbol-case preserve|downcase|upcase`
- `--comment-style preserve|cl`
- `--block-comments preserve|to-semicolons`
- `--contextual-comments yes|no`
- `--inline-comment-column N`
- `--in-place`
- `--check`
- `--output FILE`

10.2 Sémantique attendue

- `--check` retourne un statut indiquant si le fichier est déjà conforme ;
- `--in-place` remplace le fichier source par la version formatée ;
- `--output FILE` écrit vers un autre fichier ;
- l'absence de `--in-place` implique une sortie externe ou standard.

11 Stratégie de représentation interne

Le modèle interne doit séparer autant que possible :

- structure syntaxique ;
- trivia (espaces, commentaires, sauts de ligne) ;
- rendu final.

Une représentation raisonnable pour chaque nœud inclut :

- type du nœud ;
- texte brut éventuel ;
- enfants ;
- trivia préfixe ;
- trivia suffixe ;
- position source optionnelle.

12 Règles de qualité

12.1 Déterminisme

Même entrée + mêmes options = même sortie.

12.2 Idempotence

`fmt(fmt(source)) = fmt(source)`.

12.3 Non-régression syntaxique

Le programme formaté doit rester lisible par AutoLISP.

12.4 Lisibilité

Le rendu doit privilégier :

- indentation stable ;
- groupement cohérent ;
- fermeture de parenthèses régulière ;
- commentaires exploitables par les développeurs.

13 Gestion des erreurs

13.1 Erreurs détectables

- parenthèses non équilibrées ;
- chaîne non terminée ;
- commentaire bloc non terminé ;
- quote orpheline en fin de fichier ;
- token inattendu empêchant la reconstruction structurelle.

13.2 Comportement attendu

Le programme doit :

- signaler l'erreur avec position approximative si possible ;
- ne pas produire silencieusement une sortie corrompue ;
- permettre un mode strict et éventuellement un mode tolérant ultérieur.

14 Jeux de tests requis

14.1 Tests unitaires scanner

Couvrir au minimum :

- parenthèses simples ;
- commentaires de ligne ;
- commentaires bloc ;| ... | ;
- chaînes contenant parenthèses et points-virgules ;
- symboles avec caractères spéciaux ;
- `:foo` ;
- `/` dans listes de paramètres ;
- dotted pairs.

14.2 Tests unitaires parseur

- listes simples ;
- listes imbriquées ;
- formes avec quote ;
- commentaires entre formes ;
- détection d'erreurs de structure.

14.3 Golden tests pretty-printer

Pour chaque cas, comparer la sortie exacte attendue :

- style `cl` ;
- style `nail` ;
- casse préservée ;
- casse uniformisée ;
- commentaires préservés ;
- conversion de commentaires bloc.

14.4 Tests d'idempotence

Formater deux fois et vérifier l'égalité exacte des sorties.

14.5 Tests de non-régression sur corpus réel

Appliquer le formateur sur un corpus AutoLISP existant d'équipe et conserver des résultats de référence.

15 Plan de livraison recommandé

15.1 Version 1

Inclure :

- lecture fichier ;

- scanner complet ;
- parseur léger ;
- pretty-printer ;
- styles `cl` et `nail` ;
- options de casse des symboles ;
- préservation des commentaires ;
- conversion simple des commentaires bloc ;
- tests unitaires et golden tests.

15.2 Version 2

Peut inclure :

- heuristiques de commentaires plus fines ;
- davantage de règles d'indentation spécifiques ;
- meilleure gestion des lignes blanches ;
- intégration outil / éditeur / CI.

16 Critères d'acceptation

Le projet sera considéré comme acceptable si :

- il est entièrement écrit en AutoLISP ;
- il reformate correctement les parenthèses selon les deux styles ;
- il gère correctement `| ... |` ;
- il propose les trois politiques de casse des symboles ;
- il sait préserver ou normaliser les commentaires selon options ;
- il est déterministe et idempotent ;
- il passe les jeux de tests définis ;
- il est utilisable par l'équipe sur un corpus réel sans corruption du code.

17 Style supplémentaire de parenthèses : nail-clipping à fermetures empilées

There exists a particularly undesirable variant of nail-clipping where several closing parentheses are stacked on a single line. Example:

```
(defun foo (x)
  (if (> x 0)
      (progn
        (bar x)
        (baz x)
      )
  ) )
```

This style creates a visual illusion that closing parentheses correspond vertically to the opening parentheses above them, whereas their order is in fact reversed when read left-to-right. In this specification, this style shall be named:

- **nail-clipping à fermetures empilées**

Alternative informal aliases that may be mentioned in documentation:

- **nail-clipping compacté**
- **nail-clipping en chaîne**
- **fermetures empilées sur une même ligne**

The formatter shall support explicit handling of this style as follows:

- It shall detect it.
- It shall never emit it unless an explicit compatibility mode is requested.
- By default, when encountered, it shall normalize it to the selected target parenthesis style (typically classic Common Lisp style or regular nail-clipping with one closing parenthesis per line as needed).
- A diagnostic/check mode should be able to report occurrences of this style.

17.1 Parenthesis-style enumeration update

The formatter parenthesis style option shall support at least:

- `:cl`
- `:nail`
- `:stacked-nail` (compatibility / legacy input-output mode only)

Semantics:

- `:cl`: close as late as possible, on the final logical line.
- `:nail`: nail-clipping style, but with structurally separated closing parentheses, never misleadingly stacked on one line.
- `:stacked-nail`: preserve or emit the legacy stacked-closing nail-clipping style. This mode exists only for transitional compatibility and should be discouraged in team usage.

17.2 Validation rule

Unless `:paren-style :stacked-nail` is explicitly selected, the formatter output must not contain any line whose only non-whitespace content is a mixed sequence of multiple closing parentheses separated only by spaces, such as:

)))

or equivalent variants.

17.3 Naming rationale

The chosen name emphasizes the actual phenomenon:

- it is derived from nail-clipping style,
- its closers are stacked on the same physical line,
- and it is visually misleading.

18 Exceptions de casse par fichier de symboles

Dans certains projets, un petit ensemble de symboles doit conserver une capitalisation précise pour de bonnes raisons pratiques, par exemple :

- interface avec des bibliothèques externes ;
- noms de variables d’environnement ;
- étiquettes destinées à être lues par des humains ;
- conventions historiques du projet ;
- compatibilité avec des données ou des API externes.

Le formateur doit donc permettre l’usage d’un **fichier d’exceptions de capitalisation**.

18.1 Format du fichier d’exceptions

Le fichier contient un symbole par ligne, exactement tel qu’il doit être réécrit dans la sortie.

Exemple :

```
PATH
CreateFileW
LabelCourte
MaConventionLocale
```

Les lignes vides sont autorisées et doivent être ignorées. Une extension future pourra autoriser des commentaires dans ce fichier, mais ce n’est pas requis pour la première version.

18.2 Sémantique

Lorsque ce fichier est fourni :

- chaque occurrence d’un symbole du source doit être comparée aux entrées du fichier de manière insensible à la casse ;
- si un symbole du source correspond à une entrée du fichier, il doit être réécrit exactement comme dans le fichier ;

- cette réécriture est prioritaire sur la politique générale de casse des symboles.

Exemple :
si le fichier contient :

```
PATH
CreateFileW
```

alors les occurrences suivantes :

```
path
Path
createfilew
CREATEFILEW
```

doivent être réécrites en :

```
PATH
PATH
CreateFileW
CreateFileW
```

18.3 Priorité des règles de casse

L'ordre de priorité doit être le suivant :

1. exceptions de capitalisation fournies par fichier ;
2. politique générale de casse (**préserver**, **minuscules**, **majuscules**) ;
3. texte source d'origine si aucune autre règle ne s'applique.

18.4 Options associées

Les spécifications doivent prévoir au moins les options suivantes :

- chemin du fichier d'exceptions de capitalisation ;
- activation ou désactivation explicite de cette fonctionnalité ;
- mode de diagnostic permettant de signaler les symboles présents dans le fichier d'exceptions mais jamais rencontrés dans le source.

19 Fichier de configuration du formateur

Toutes les options du programme doivent pouvoir être fournies soit directement, soit au moyen d'un **fichier de configuration** passé comme seul argument au programme ou à la fonction principale de formatage.

19.1 Objectif

Cette fonctionnalité permet :

- d'éviter des lignes de commande trop longues ;
- de versionner une politique de formatage commune à l'équipe ;
- de partager facilement les mêmes réglages entre l'outil en ligne de commande et l'API AutoLISP ;
- d'inclure les chemins de fichiers auxiliaires, notamment le fichier d'exceptions de capitalisation.

19.2 Formes acceptées

Le fichier de configuration peut contenir soit :

1. une seule s-expression représentant la liste complète des options et arguments ;
2. ou bien directement les options et arguments, tels quels, sans liste englobante.

Exemples acceptés :

```
(:style-parentheses :cl
:casse-symboles :minuscules
:fichier-exceptions-casse "capitalisation.txt"
:colonne-commentaires-en-ligne 40
"entree.lsp" "sortie.lsp")

:style-parentheses :cl
:casse-symboles :minuscules
:fichier-exceptions-casse "capitalisation.txt"
:colonne-commentaires-en-ligne 40
"entree.lsp" "sortie.lsp"
```

Dans le second cas, l'implémentation peut lire tout le contenu et l'encapsuler logiquement dans une liste avant analyse.

19.3 Correspondance programme / API

Le même schéma d'options doit être accepté :

- par le programme appelé en ligne de commande ;
- par la fonction AutoLISP de formatage ;
- par le fichier de configuration.

L'objectif est d'avoir une source unique de vérité pour l'interprétation des options.

19.4 Résolution des conflits

Si plusieurs modes de fourniture des options coexistent, la priorité recommandée est :

1. options explicitement passées à l'appel courant ;
2. options lues dans le fichier de configuration ;
3. valeurs par défaut.

Toute implémentation doit documenter clairement cette règle et l'appliquer de manière déterministe.

19.5 Options minimales à prévoir

Les spécifications doivent désormais inclure, en plus des options déjà prévues :

- option pour charger un fichier de configuration ;
- option pour charger un fichier d'exceptions de capitalisation ;
- option pour choisir le mode de casse générale des symboles ;
- option pour activer un contrôle de cohérence des exceptions de capitalisation ;
- option permettant de faire traiter un contenu de configuration comme une seule s-expression ou comme une suite d'éléments à regrouper implicitement.

19.6 Recommandation de mise en oeuvre

L'analyse des options doit être factorisée dans un module unique, utilisé à la fois par :

- l'interface de ligne de commande ;
- le lecteur de fichier de configuration ;
- l'appel programmatique depuis AutoLISP.

Ainsi, toutes les variantes d'entrée produisent la même structure interne d'options normalisées.